

__tmp__reducido

May 29, 2026

1 Modelos estocásticos y redes neuronales

Materia: Análisis de Series de Tiempo y Pronósticos (1C-2026)

Grupo: 9

Integrantes:

- Lucas Achaval - Email: lachavalrodriguez@estudiantes.unsam.edu.ar
- Marcos Achaval - Email: machavalrodriguez@unsam-bue.edu.ar

Título del entregable: Modelos estocásticos y redes neuronales

Conjunto de datos original: dataset propio de una estación meteorológica en un club náutico de Potrerillos (disponibles en [Windguru](#)).

1.1 Resumen

Este trabajo busca construir un modelo de pronóstico horario de viento promedio (`wind_avg`) en Potrerillos, Mendoza, con horizonte de 12 horas, evaluado mediante MAE, RMSE y R^2 . En la primera entrega caracterizamos la serie (ciclo diurno marcado, estacionariedad confirmada por ADF, picos de ACF/PACF en lags 1-2 y 20-24) y entrenamos un LassoCV con 24 lags, 6 exógenas y codificación cíclica de la hora, que redujo el RMSE **aproximadamente un 25 %** y elevó el R^2 de 0.117 a 0.498 frente a una persistencia estacional, sin signos de sobreajuste. Esta entrega avanza con benchmark, modelos estocásticos y redes neuronales.

1.2 Carga y preprocesamiento

Replicamos el preprocesamiento del primer entregable: lectura del archivo crudo, remuestreo horario (mediana para variables lineales, media circular para `wind_direction`) y agregado del atributo auxiliar `interval`. La variable `wind_min` queda fuera porque está completamente vacía, y `gustiness` se omite porque es una derivada de `wind_avg` y `wind_max`.

1.2.1 Split de entrenamiento y test

Antes de entrenar cualquier modelo separamos un **80 % inicial cronológico** como entrenamiento y reservamos el **20 % final** como test. Toda la búsqueda de hiperparámetros (orden de SARIMA, arquitectura de las NN) y todos los diagnósticos se hacen exclusivamente sobre el train; el test se reserva para reportar el desempeño final del benchmark, SARIMA y las redes neuronales **bajo la misma metodología de evaluación**.

Los 276 valores faltantes (4.5 % de la serie) los completamos con `ffill().bfill()`, igual que en la descomposición estacional del primer entregable. Definimos también las constantes globales `HORIZONTE = 12` (pasos a predecir) y `PERIODO = 24` (estacionalidad horaria).

1.3 Modelo de referencia (benchmark)

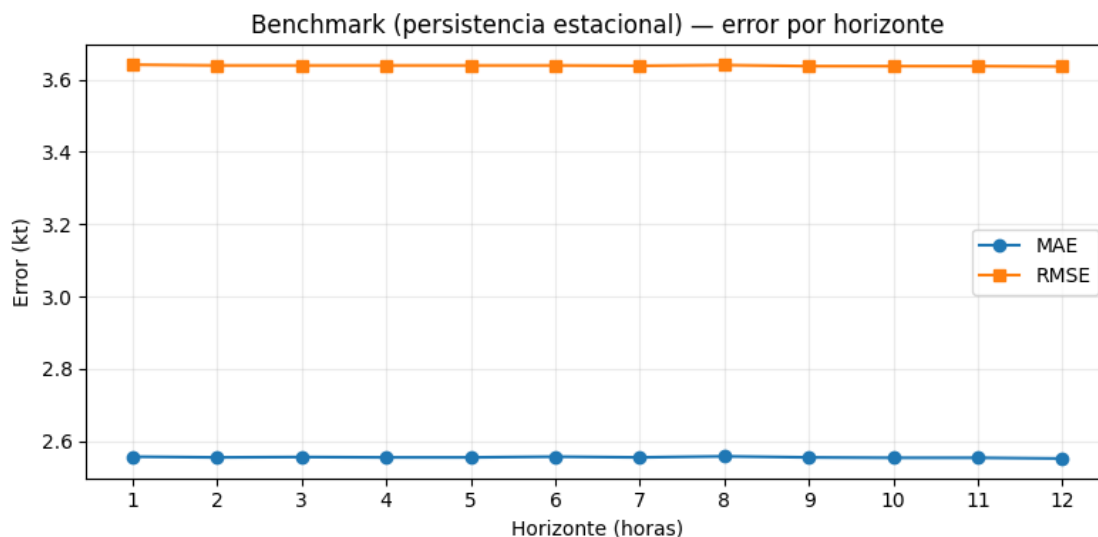
Como benchmark adoptamos la **persistencia estacional** con período 24 h. Para un instante t y un horizonte $h \leq 24$:

$$\hat{y}_{t+h} = y_{t+h-24}$$

Es decir, el viento de cada hora se pronostica con el valor observado a la misma hora del día anterior. Justificación de la elección:

- **Simple y de interpretación inmediata.** No tiene parámetros a estimar.
- **Aprovecha la única estructura fuerte de la serie.** El ACF del primer entregable mostró un pico claro en el lag 24 (autocorrelación ≈ 0.5). Una persistencia simple ($\hat{y}_{t+h} = y_t$) ignoraría esa estacionalidad y rendiría peor.
- **Línea de base no trivial.** Cualquier modelo más complejo deberá superar este desempeño para justificar su costo.

Lo evaluamos sobre `serie_test` con un esquema **walk-forward**: para cada origen t del test producimos los pronósticos a 1, 2, ..., 12 horas y los contrastamos con las observaciones reales. Es la misma metodología que aplicaremos a SARIMA y a las redes neuronales, de modo que las tablas son directamente comparables.



1.3.1 Desempeño del benchmark

La tabla y el gráfico anteriores muestran MAE, RMSE y R^2 del benchmark para cada horizonte de 1 a 12 horas. Observaciones:

- **Error casi constante con el horizonte.** Como la predicción siempre se construye con un dato de 24 h atrás, no importa cuán lejos esté el target dentro de las próximas 24 horas. El error se mantiene en el mismo orden. Este perfil plano será un contraste útil frente a modelos con memoria de corto plazo (SARIMA, redes neuronales), que tienden a degradarse a medida que se aleja el horizonte.
- **R² positivo pero bajo.** La estacionalidad diaria sola explica una fracción modesta de la varianza del viento, dejando margen amplio para que los modelos posteriores aporten información adicional vía lags cercanos y variables exógenas.

A partir de aquí, este será el desempeño mínimo que deben superar los modelos estocásticos y de redes neuronales para resultar útiles.

1.4 Modelo estocástico

A partir de las funciones de autocorrelación (ACF) y autocorrelación parcial (PACF) buscamos un modelo estocástico que ajuste bien la dinámica de `wind_avg`. El primer entregable ya mostró tres hechos relevantes para esta etapa:

- La prueba ADF sobre la serie original arrojó un estadístico de -11.4 y un p -valor de 7.6×10^{-21} , rechazando con holgura la hipótesis de raíz unitaria. **La serie es estacionaria.**
- El ACF mostró un pico claro en el lag 24 y el PACF resaltó los lags 1-2 y 20-24.
- El espectro de potencias estuvo dominado por la frecuencia de 1 ciclo/día, con un segundo pico (más débil) en 2 ciclos/día.

1.4.1 ¿Por qué SARIMA y no AR, MA, ARMA o ARIMA?

La materia presenta la familia de modelos en escalera: AR, MA, ARMA (estacionarios), ARIMA (no estacionarios por tendencia, vía diferenciación regular) y SARIMA (cuando además hay estructura estacional). La elección entre ellos no es discrecional, se desprende de qué muestre la serie. Las guías 4 y 5 establecen las reglas:

Modelo	ACF esperado	PACF esperado
AR(p)	decae	corta en lag p
MA(q)	corta en lag q	decae
ARMA(p, q)	decae	decae

- **AR puro queda descartado.** La PACF de `wind_avg` no corta limpio: tiene un primer cluster de 2 lags significativos (1-2) y otro cluster alrededor de los lags 20-24. Para que un AR(p) capture el pico de lag 24 con la PACF cortando ahí, haría falta $p = 24$ (24 coeficientes), violando el criterio de parsimonia.
- **MA puro queda descartado.** El ACF no corta en ningún lag corto: decae con oscilaciones (efecto día/noche en lag 12, pico positivo en lag 24). Un MA(q) requiere que la ACF se anule a partir de cierto q , condición que aquí no se cumple.
- **ARMA estacional-agnóstico queda descartado.** ARMA puede modelar un ACF/PACF que decaen, pero no incorpora la noción de **período**. El pico aislado en lag 24 de la ACF es estacional, no una correlación corta extendida, y un ARMA tendría que aproximarla con muchos coeficientes consecutivos. El espectro de potencias confirma que el ciclo diario es la fuente dominante de varianza, no ruido autocorrelacionado de corto alcance.

- **ARIMA queda descartado porque no hace falta diferenciación regular.** El ADF confirma estacionariedad, así que $d = 0$ y ARIMA(p,0,q) colapsa al ARMA ya descartado.
- **SARIMA es el ajuste natural.** Modela explícitamente la estacionalidad con un solo término P o Q aplicado al rezago $\mathbf{B}^{24}$, según la guía 5:

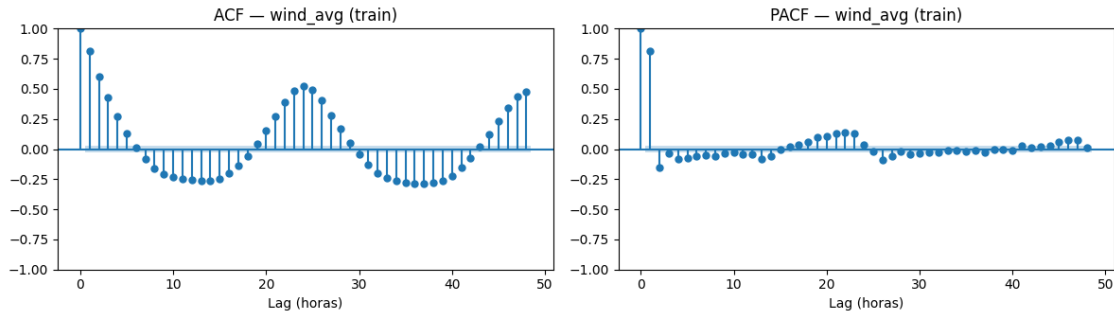
$$\Theta_P(\mathbf{B}^{24}) \Theta_p(\mathbf{B}) (1 - \mathbf{B}^{24})^D (1 - \mathbf{B})^d x_t = \Phi_Q(\mathbf{B}^{24}) \Phi_q(\mathbf{B}) w_t$$

Así un único coeficiente estacional reemplaza decenas de coeficientes no estacionales. Tomamos $d = 0$ (ADF) y, en principio, $D = 0$ porque el ADF también pasa sobre la serie no diferenciada y el pico estacional ya está presente sin necesidad de remover una tendencia estacional. Si los residuos del modelo final muestran estructura sobreviviente cerca de múltiplos de 24, reconsideramos $D = 1$.

El modelo a explorar es entonces un **SARIMA(p,0,q)(P,0,Q,24)**, dejando para la búsqueda con AIC la elección concreta de p, q, P, Q . Toda la búsqueda y el diagnóstico se hacen sobre `serie_train` (el 80 % inicial ya definido arriba).

1.4.2 Funciones de autocorrelación sobre el training

Repetimos el análisis del entregable 1 pero **solo sobre el subconjunto de training**, para evitar mirar el test al elegir los hiperparámetros. Esto es importante metodológicamente: cualquier inspección que hagamos para definir el modelo (orden, estacionalidad) debe basarse exclusivamente en datos vistos.



El comportamiento sobre el training reproduce lo observado en el entregable 1: PACF significativa en lags 1-2 y entre 20-24, ACF con decaimiento lento y pico positivo en lag 24. Esto confirma la elección de SARIMA argumentada arriba y permite definir la grilla a explorar:

- $p \in \{0, 1, 2\}$ (PACF significativa hasta lag 2)
- $q \in \{0, 1\}$ (MA corto opcional)
- $P, Q \in \{0, 1\}$ (un solo término estacional alcanza para capturar el pico de lag 24)
- $d = D = 0, s = 24, \text{trend} = 'c'$

Son $3 \times 2 \times 2 \times 2 = 24$ combinaciones, en línea con el criterio de **parsimonia**.

1.4.3 Selección del orden con el criterio de Akaike (AIC)

Iteramos sobre la grilla y, para cada combinación, ajustamos un SARIMA con `statsmodels.tsa.arima.model.ARIMA` y registramos el AIC. El AIC se define como

$$\text{AIC} = -2\log \mathcal{L} + 2k$$

donde $\mathcal{L}$ es la verosimilitud del modelo y k es el número de parámetros. Penaliza la cantidad de parámetros, por lo que minimizarlo selecciona el modelo más simple compatible con los datos (criterio de parsimonia). Envolvemos el ajuste en `try/except` para descartar combinaciones que no converjan, igual que en la guía 5.

La tabla anterior ordena las 24 combinaciones por AIC creciente. Tomamos como modelo ganador la primera fila (AIC mínimo).

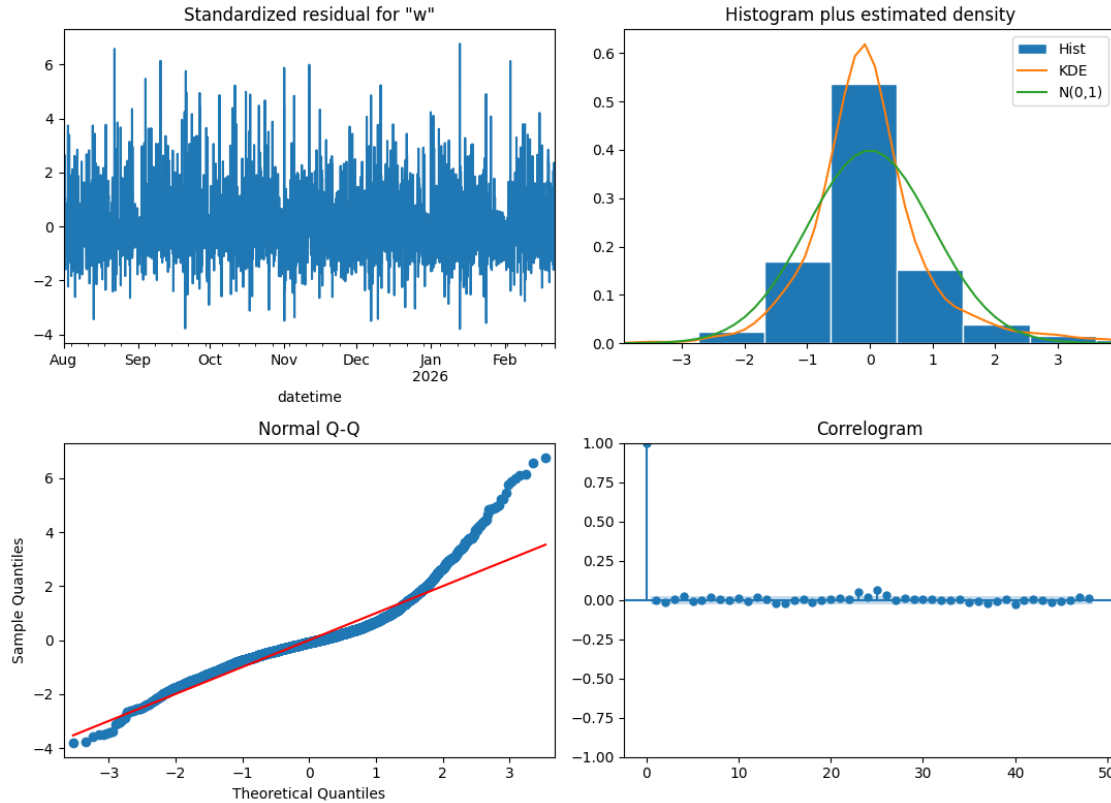
1.4.4 Ajuste del modelo ganador

Refitteamos el orden ganador para inspeccionar los coeficientes con `summary()`: nos interesa ver si todos los términos son estadísticamente significativos (columna $P > |z|$ cerca de cero) y si las raíces de los polinomios caen fuera del círculo unitario (condición de estacionariedad e invertibilidad).

1.4.5 Diagnóstico de residuos

Para confirmar que el orden elegido es adecuado revisamos los residuos: si el modelo captura toda la estructura, los residuos deben comportarse como un ruido blanco (independientes, idénticamente distribuidos, media cero). Usamos `plot_diagnostics()` de `statsmodels`, el panel que la guía 4 introduce y que agrupa:

1. **Residuos estandarizados en el tiempo:** no debería verse tendencia, heterocedasticidad ni outliers sistemáticos.
2. **Histograma + KDE comparados con $\mathcal{N}(0, 1)$:** los residuos deberían ajustarse razonablemente a una normal.
3. **Q-Q plot:** los puntos deberían alinearse sobre la diagonal; desvíos en las colas indican distribuciones de cola pesada.
4. **Correlograma de los residuos:** todos los lags deberían quedar dentro de la banda de confianza, sin estructura sobreviviente.



Interpretación del diagnóstico:

- **Residuos estandarizados en el tiempo:** oscilan alrededor de 0 sin tendencia ni cambio sistemático de varianza. Se ven algunos picos aislados de magnitud 4-7 (outliers) repartidos a lo largo de toda la serie, sin formar clusters que sugieran heterocedasticidad fuerte.
- **Histograma + KDE vs $\mathcal{N}(0,1)$:** la forma de la distribución empírica (KDE en naranja) no coincide con la normal de referencia (verde): hay más masa concentrada cerca de 0 (pico más alto) y más masa en las colas (al menos en la derecha), con menos masa en la zona intermedia. Sin embargo, las dos cosas se compensan y la varianza total se mantiene cerca de 1, pero la forma no es gaussiana.
- **Q-Q plot:** los puntos se alinean con la diagonal en el centro y en la cola izquierda (de -3 a -1 la desviación es mínima). El problema está en la cola derecha: a partir de $+1.5$ los *Sample Quantiles* se elevan muy por encima de la diagonal (llegan a 6-7 cuando los *Theoretical Quantiles* correspondientes son solo 3). Esto indica asimetría positiva: el modelo predice bien los vientos bajos y normales pero subestima los picos altos de *wind_avg* (rachas, eventos térmicos intensos). Tiene sentido físico porque la distribución del viento es asimétrica (no puede ser muy negativa, pero puede subir mucho) y los residuos heredan esa asimetría.
- **Correlograma:** la gran mayoría de los lags del 1 al 48 caen dentro de la banda de confianza del 95%. Hay un par de lags (23 y 25) que asoman apenas por encima de la banda. Con 48 lags graficados es esperable por puro azar estadístico que algunos pocos caigan fuera del 95% aunque los residuos sean ruido blanco perfecto, así que estos excesos chicos justo en la zona estacional no justifican complicar el modelo. La lectura general es que el SARIMA absorbió

la estructura temporal y la selección del orden (p, q, P, Q) obtenida por AIC queda validada.

El modelo es válido para pronóstico puntual de `wind_avg`: la condición de ruido blanco (autocorrelación nula) se cumple, que es la hipótesis crítica del ajuste por máxima verosimilitud. La no-normalidad de los residuos (en el Q-Q plot) es esperable en series meteorológicas (eventos de viento extremos, rachas atípicas) y no invalida los coeficientes estimados; sin embargo, debe tenerse en cuenta al construir intervalos de confianza para las predicciones, ya que los basados en la hipótesis gaussiana van a subestimar la probabilidad de eventos extremos.

Con el modelo seleccionado y validado por residuos, queda definido el modelo estocástico de referencia. En la próxima sección lo evaluaremos sobre `serie_test` con un walk-forward idéntico al del benchmark y compararemos su desempeño contra la persistencia estacional.

1.5 Evaluación del modelo estocástico

1.5.1 Metodología (windowing)

Evaluamos SARIMA con la misma estrategia que usamos para el benchmark. Ajustamos el modelo una vez sobre `serie_train` y guardamos `serie_test` para reportar métricas. La guía 5 muestra `extend()` como forma de evaluar predicciones a 1 paso sobre el test, pero la consigna pide métricas según el horizonte (hasta $h = 12$), así que extendemos esa idea de forma iterativa con un **walk-forward** sobre `serie_test`.

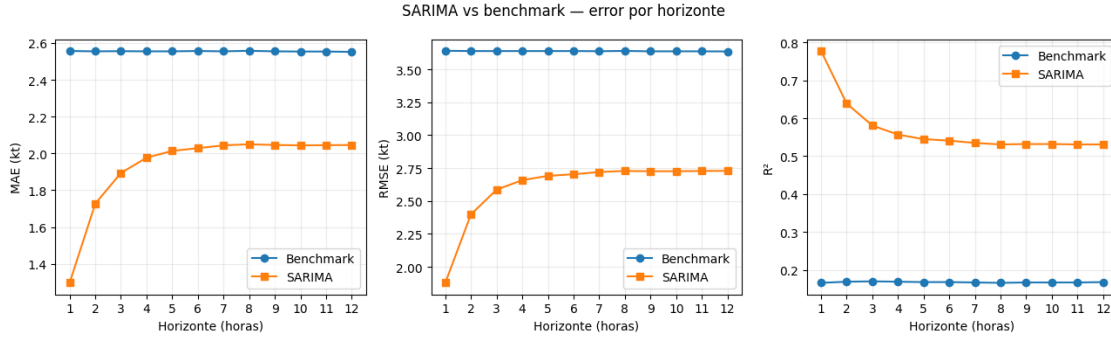
El procedimiento avanza origen por origen del test.

1. Para cada origen t producimos un pronóstico a 12 h con `forecast(steps=12)` y registramos $\hat{y}_{t+1}, \dots, \hat{y}_{t+12}$.
2. Antes de avanzar al siguiente origen, incorporamos la observación recién pronosticada al estado del modelo con `append(refit=False)`, **sin re-estimar coeficientes**. Así dejamos el filtro de Kalman al día con el último dato observado, igual que pasaría en operación. Es la versión iterativa del `extend()` que aparece en la guía 5. En vez de extender el modelo de una sola vez sobre todo el test, lo extendemos paso a paso para que cada predicción solo use información hasta el instante anterior.

Este patrón `forecast(...) → append(refit=False)` es el que recomienda la documentación de `statsmodels` para evaluación walk-forward eficiente, y es la extensión natural de la guía 5 a pronósticos multi-paso.

¿Por qué no re-fitear en cada origen? Re-estimar parámetros en cada uno de los ~1200 orígenes implicaría ajustar SARIMA estacional ($s = 24$) más de mil veces, con un costo computacional enorme para una mejora marginal. Lo habitual es entrenar al inicio y solo actualizar el estado dentro del test.

Calculamos MAE, RMSE y R^2 por horizonte con la misma función `metricas_por_horizonte` que usamos para el benchmark, así ambas tablas quedan directamente comparables.



1.5.2 Interpretación de los resultados

La tabla anterior y los gráficos comparan SARIMA contra el benchmark en MAE, RMSE y R^2 horizonte por horizonte. Lo más relevante.

- **SARIMA supera al benchmark en todos los horizontes.** Las curvas no se cruzan. Incluso en el horizonte más largo el RMSE de SARIMA queda por debajo del benchmark, y el R^2 del modelo estocástico es varias veces mayor que el de la persistencia estacional.
- **Degradación marcada en los primeros pasos y meseta a partir de h 6.** El RMSE crece rápido desde $h = 1$ y después se estanca. Esto se entiende mirando la estructura del modelo. El bloque AR(2)/MA(1) tiene raíces dentro del círculo unitario que decaen rápido (~5-6 pasos), mientras que el bloque estacional $(1, 0, 1, 24)$, con `ar.S.L24` muy cercano a 1, se comporta como un patrón cíclico estable. A horizontes largos la predicción converge esencialmente a la media condicional más el ciclo diario, y de ahí no baja más.
- **La meseta queda por encima del benchmark.** A horizontes lejanos SARIMA y benchmark usan la misma información (el ciclo de 24 h), pero SARIMA trabaja con una estimación suavizada del ciclo, mientras que el benchmark copia un único dato ruidoso de ayer. Esa diferencia de varianza explica la brecha que queda entre las dos curvas.

Con esto queda evaluado el modelo estocástico. La línea de base sube bastante respecto al benchmark, así que cualquier red neuronal que entrenemos en la próxima sección va a tener que superar estas cifras para justificar su complejidad.

1.6 Modelo de redes neuronales

Para el pronóstico de `wind_avg` a 12 h armamos modelos de redes neuronales siguiendo la guía 7 del curso (LSTM endógeno). A diferencia de SARIMA, una red neuronal no asume un proceso lineal y aprende una función cualquiera desde una ventana de pasos pasados hacia los 12 pasos futuros. Eso obliga a agregar dos pasos de preprocesamiento que no hacían falta en SARIMA.

- **Normalización** de la entrada, para que el optimizador trabaje en una escala controlada y los pesos no exploten en la primera época.
- **Ventaneo**, para transformar la serie temporal continua en pares (X_i, y_i) que la red pueda consumir como muestras independientes.

Mantenemos `serie_train` y `serie_test` (con el split 80/20 ya definido) y conservamos el horizonte de 12 h y la frecuencia horaria. Para cubrir lo que pide la consigna probamos **tres configura-**

ciones, una red densa de referencia, una LSTM simple con dropout y una LSTM apilada con `return_sequences=True`.

1.6.1 Normalización y ventaneo

Normalización Z-score con estadísticos del train. Restamos la media y dividimos por la desviación estándar calculadas **solo sobre `serie_train`**, y aplicamos esa misma transformación a `serie_test`. Es la práctica estándar de la guía 7. Así evitamos filtrar información del test al ajuste y dejamos la entrada centrada en 0 con varianza 1. Aprovechamos para castear a `float32`, el `dtype` por defecto de Keras.

Ventaneo sequence-to-vector con salida multi-paso. Recorremos la serie con paso 1 y, para cada origen i , formamos los pares siguientes.

- $X_i = (x_i, x_{i+1}, \dots, x_{i+W-1})$ — los W valores pasados,
- $y_i = (x_{i+W}, x_{i+W+1}, \dots, x_{i+W+H-1})$ — los $H = 12$ valores futuros que queremos predecir.

Tomamos $W = 48$ para meter **dos ciclos diarios completos** en la ventana de entrada, siguiendo la lógica de la guía 7 de cubrir varios ciclos estacionales y darle a la red información estructural suficiente. La salida tiene dimensión $H = 12$. Configuramos la última capa densa con 12 unidades para predecir los 12 horizontes **en un solo paso** (*direct multi-step*, ejercicio 7 de la guía). Es más simple que un esquema recursivo (ejercicio 6) y evita el error acumulado de realimentar predicciones, lo que además deja la comparación con SARIMA más prolija.

1.6.2 Arquitecturas propuestas

Las tres configuraciones comparten el bloque de entrada (`Input((48, 1))`) y la cabeza de salida (`Dense(12, activation='linear')`), y se diferencian en lo que pasa en el medio.

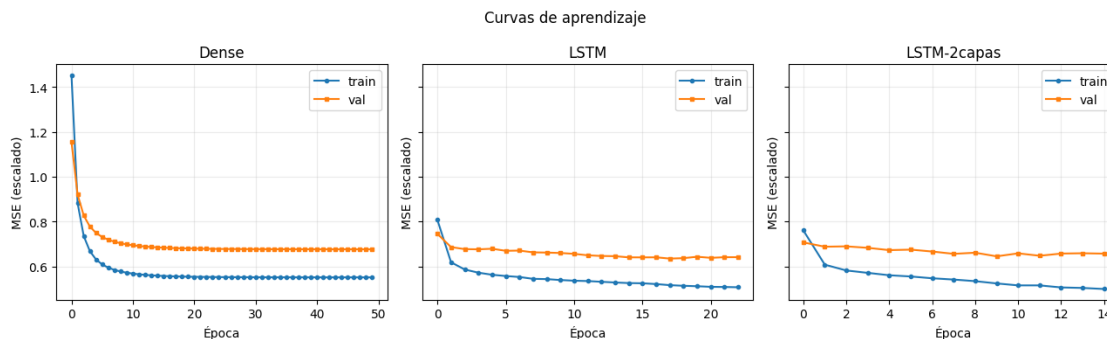
- **Dense:** red de referencia tipo guía 7. `Flatten` aplana la ventana de 48 pasos a un vector de 48 valores y un único `Dense(12)` aprende coeficientes lineales hacia cada horizonte. En el fondo es una regresión lineal multi-objetivo sobre los lags, y sirve para medir cuánto aporta la estructura recurrente cuando la agreguemos.
- **LSTM:** una capa `LSTM(32)` recorre la ventana paso a paso y entrega el estado oculto final como vector de 32 features, seguida de `Dropout(0.2)` (durante el entrenamiento desactiva al azar el 20 % de las unidades para regularizar y evitar coadaptación) y el `Dense(12)` de salida. Es la arquitectura típica de la guía 7.
- **LSTM-2capas:** apila dos LSTM. La primera usa `return_sequences=True` para entregar la secuencia completa de estados ocultos (no solo el último), así la segunda LSTM puede operar sobre esa secuencia, como pide el ejercicio 5 de la guía 7. Cada LSTM va seguida de su `Dropout(0.2)`. Sirve para ver si más profundidad ayuda a capturar dinámica no lineal adicional.

El optimizador es Adam y la pérdida es MSE (igual que en la guía 7). Fijamos la semilla de Keras a 99 para que los pesos iniciales y el dropout sean reproducibles entre corridas.

1.6.3 Entrenamiento

Entrenamos cada modelo sobre `X_train_nn` con `validation_split=0.1` (último 10 % del train como validación, manteniendo el orden temporal) y un `EarlyStopping` con `patience=5` sobre la

pérdida de validación que restaura los mejores pesos cuando se corta. Es el patrón sugerido en el ejercicio 4 de la guía 7. Dejamos `epochs=50` como tope y la convergencia decide cuándo parar.



1.6.4 Resumen del ajuste

Modelo	Parámetros	Épocas ajustadas	<code>val_loss</code> mínima
Dense	588	50 (tope)	0.676
LSTM	4 748	23	0.635
LSTM-2capas	13 068	15	0.645

Algunas lecturas de las curvas de aprendizaje.

- **Las dos LSTM dispararon EarlyStopping antes del tope** (23 y 15 épocas), señal de que la pérdida de validación dejó de mejorar y los pesos restaurados son los del mínimo histórico. La Dense llegó al tope de 50 épocas **sin** disparar **EarlyStopping**. Con esa cota su pérdida seguía bajando despacio, así que el 0.676 reportado es una cota superior y no necesariamente refleja el mínimo que alcanzaría con más épocas.
- **La LSTM simple obtiene la menor `val_loss`** (0.635 contra 0.645 de la apilada y 0.676 de la densa). La diferencia con la apilada (0.010) sale de **una sola semilla** (`seed = 99`) y puede estar dentro del ruido entre corridas. Una comparación más sólida exigiría promediar varias semillas y reportar $\text{media} \pm \text{desvío}$, lo que no hicimos en esta entrega. Con esta única corrida lo que se puede afirmar es que **las dos LSTM quedan claramente por debajo de la Dense**, y entre LSTM y LSTM-2capas la diferencia no es concluyente.
- **Las curvas de `train` y `validación` divergen poco**, lo que indica que el `dropout = 0.2` cumple su rol de regularización y no estamos sobreajustando de forma agresiva.

Las pérdidas están en la **escala normalizada** y promediadas sobre los 12 horizontes, así que no son directamente comparables con el RMSE en kt del benchmark o SARIMA. En la sección siguiente las redes pasan por el mismo walk-forward que aplicamos a SARIMA sobre `serie_test`, y eso nos permite reportar MAE, RMSE y R^2 por horizonte en escala kt.

1.7 Evaluación de la red neuronal

1.7.1 Modelo elegido y metodología

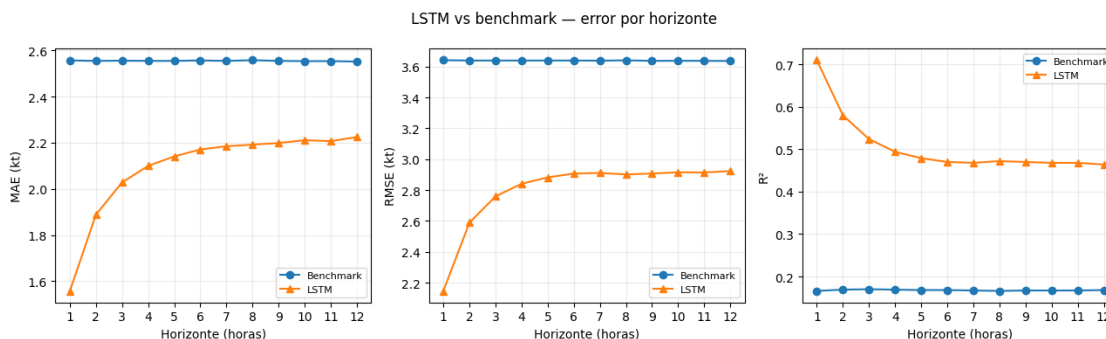
Del punto anterior elegimos la **LSTM simple**: obtuvo la menor `val_loss` (0.635 contra 0.645 de la apilada y 0.676 de la densa) y, frente a la LSTM de dos capas, alcanza ese desempeño con menos de la mitad de parámetros (4 748 vs 13 068). Por parsimonia nos quedamos con la más chica.

La evaluamos sobre `serie_test` con la **misma metodología que SARIMA**: un walk-forward que reporta MAE, RMSE y R^2 por horizonte (1 a 12 h) en kt, contrastado contra el **benchmark**, la línea de base que todo modelo debe superar. La comparación frente a SARIMA y la elección del modelo para una fase operativa quedan para el punto 8.

A diferencia de SARIMA, el formato es *direct multi-step*: una sola pasada de `predict()` sobre las ventanas de test (`X_test_nn`) devuelve los 12 horizontes a la vez. Cada fila j de `X_test_nn` es un origen t_j cuya predicción usa exclusivamente los 48 valores previos, así que metodológicamente equivale al loop `forecast` $\rightarrow$ `append(refit=False)` de SARIMA, sin recorrer orígenes a mano.

Dos detalles operativos:

- **Desnormalización antes de medir.** El modelo trabaja en escala z (Z-score con `train_mean` y `train_std`), así que recuperamos kt con `y * train_std + train_mean` antes de calcular las métricas, dejando las tablas comparables con benchmark y SARIMA.
- **Offset respecto de SARIMA.** El ventaneo del test arranca en `serie_test.iloc[WINDOW_SIZE + h - 1]` y SARIMA en `serie_test.iloc[h - 1]`: la LSTM se evalúa sobre 1170 orígenes y SARIMA sobre 1218. La diferencia es chica y los promedios siguen siendo comparables, pero lo dejamos asentado.



1.7.2 Interpretación de los resultados

La tabla y los gráficos comparan la LSTM contra el benchmark horizonte por horizonte.

- **La LSTM supera al benchmark en todos los horizontes.** En $h = 1$ el salto es grande: RMSE 2.15 kt contra 3.64 del benchmark y R^2 0.71 contra 0.17. La red aprovecha la información de los lags cercanos de la ventana de 48 h, algo que la persistencia estacional (un único dato de 24 h atrás) no puede hacer.
- **Degradación marcada en los primeros pasos y meseta a partir de $h = 6$.** El RMSE crece de 2.15 a ~ 2.9 kt entre $h = 1$ y $h = 6$ y después se estanca (el R^2 baja de 0.71 a ~ 0.47).

Es el mismo perfil que SARIMA: la memoria de corto plazo aporta mucho en los primeros pasos y, a horizonte lejano, la predicción converge esencialmente al ciclo diario.

- **La meseta queda por debajo del benchmark.** Incluso en $h = 12$ la LSTM (RMSE 2.92 kt, R^2 0.46) le gana a la persistencia estacional (RMSE 3.64, R^2 0.17). A horizontes largos ambos se apoyan sobre todo en el ciclo de 24 h, pero la LSTM trabaja con una versión suavizada y no con un único dato ruidoso de ayer.

La LSTM elegida supera con claridad la línea de base. En el punto 8 la comparamos contra SARIMA y decidimos qué modelo llevaríamos a una fase operativa.

1.7.3 ¿Los coeficientes son interpretables como importancia de atributos?

No de forma directa. Las tres redes entrenadas reúnen entre 588 (Dense) y 13 068 (LSTM-2capas) pesos, y la relación entre cada peso y “qué tan importante es un lag” no es la misma que en una regresión lineal.

- **Dense** es el único caso en el que los 588 pesos del único Dense ($48 \rightarrow 12$) mapean *literalmente* $\{\text{lag de entrada}\} \times \{\text{horizonte de salida}\}$, así que en principio se podrían leer como una matriz de coeficientes (es una regresión lineal multi-objetivo). Pero incluso ahí la magnitud del peso no equivale a importancia. La entrada está normalizada (Z-score) y los 48 lags están **fuertemente correlacionados entre sí** (autocorrelación alta en lags 1-2 y 24), lo que reparte el “crédito” entre lags equivalentes y hace que los pesos individuales sean engañosos como ranking de importancia.
- **LSTM y LSTM-2capas** no admiten esa lectura. Los pesos de cada celda viven en matrices de forma $(d_{\text{in}} + d_h) \times 4d_h$ que mezclan no linealmente input, estado oculto y los cuatro *gates* (input, forget, cell, output) a lo largo de los 48 pasos temporales. No hay correspondencia uno-a-uno entre un peso y un lag de entrada. La “importancia” de un lag depende de toda la trayectoria de estados ocultos, no de un coeficiente aislado.

1.8 Redes neuronales con variables exógenas

Hasta acá la red trabajó **solo con el pasado de wind_avg**. Ahora incorporamos variables exógenas siguiendo la guía 8 (*LSTM-exo*): cada paso de la ventana deja de ser un escalar y pasa a ser un vector con el target más los atributos meteorológicos, de modo que la entrada de la LSTM tiene forma $(48, n_{\text{features}})$ en lugar de $(48, 1)$. La capa recurrente recorre la ventana viendo simultáneamente la evolución conjunta de todas las variables.

Atributos elegidos. Usamos las **mismas cinco exógenas que el Lasso** (`wind_max`, `temperature`, `rh`, `mslp`, `wind_direction`). Así la LSTM-exo queda directamente comparable contra dos referencias: la **LSTM univariante** (misma arquitectura, sin exógenas) y el **Lasso** (mismo conjunto de atributos, modelo lineal). La pregunta es la misma que nos hicimos con el Lasso: *¿la información exógena le aporta algo a la red, o wind_avg ya contiene casi toda la señal predecible?*

Decisiones de diseño (deliberadamente mínimas, para aislar el efecto de las exógenas):

- **Reutilizamos la arquitectura ganadora** (LSTM(32) $\rightarrow$ Dropout(0.2) $\rightarrow$ Dense(12)) y cambiamos *solo* el ancho de la capa de entrada. Cualquier diferencia de desempeño es atribuible a las exógenas, no a la arquitectura.
- **Solo usamos valores pasados** dentro de la ventana. No hace falta conocer las exógenas futuras: en el origen del pronóstico la red ve las últimas 48 h de las seis variables y proyecta

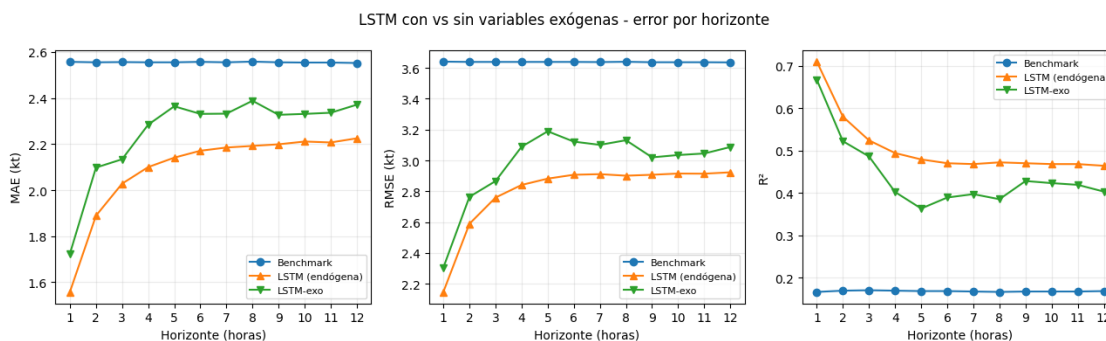
los 12 pasos de `wind_avg`. Es el mismo conjunto de información que tiene el Lasso en el origen, así que la comparación es justa y no hay fuga de datos.

- **Normalización Z-score por atributo** con estadísticos del train (igual que en la guía 8). Desnormalizamos la salida con la media y el desvío de `wind_avg` antes de medir, para reportar en kt.
- *Salvedad:* `wind_direction` es circular ($0^\circ/360^\circ$ son el mismo punto) y el Z-score no lo respeta; lo dejamos crudo por consistencia con el Lasso. Codificarla como seno/coseno sería la mejora natural, pero la mantenemos simple.

Construcción de las ventanas y la arquitectura. La función `ventanas_mv` arma, para cada origen, una ventana (48, 6) con las seis variables y un target (12,) que toma **solo** los 12 valores futuros de `wind_avg` (columna 0). `build_lstm_exo` es idéntica a la LSTM ganadora salvo por `Input((48, 6))`. Entrenamos con el mismo protocolo: `validation_split=0.1`, `EarlyStopping(patience=5)` con restauración de los mejores pesos, semilla 99.

1.8.1 Evaluación de la LSTM-exo

La evaluamos con la **misma metodología** que la LSTM univariante: una pasada de `predict()` sobre las ventanas de test, desnormalización a kt y métricas por horizonte (1 a 12 h). Como comparte ventana ($W = 48$) y horizonte ($H = 12$) con la LSTM endógena, ambas se miden sobre exactamente los mismos orígenes, así que la comparación entre las dos es directa.



1.8.2 Interpretación de los resultados

La tabla compara la LSTM-exo contra la LSTM univariante y el benchmark, horizonte por horizonte. El resultado es nítido y, en cierto modo, aleccionador:

- **Las exógenas mejoran el ajuste pero empeoran la generalización.** Durante el entrenamiento la LSTM-exo alcanzó una *val_loss menor* que la univariante (0.567 contra 0.635): con seis canales de entrada la red tiene más con qué ajustar el tramo de validación. Sin embargo, **sobre el test el orden se invierte**: la LSTM-exo es peor en todos los horizontes (RMSE promedio 2.98 kt contra 2.80 de la univariante, R^2 0.44 contra 0.51). Ya en $h = 1$ pierde (RMSE 2.31 contra 2.15 kt).
- **Es un caso de libro de sobreajuste.** Las cinco exógenas suman parámetros y grados de libertad que la red aprovecha para memorizar relaciones del train/validación que **no se sostienen** en el test. `wind_max` está fuertemente correlacionada con el target, y `temperature`,

`rh`, `mslp` y `wind_direction` aportan sobre todo ruido a esta resolución horaria: en lugar de información robusta, agregan varianza.

- **Coincide con lo que veremos en el Lasso.** El modelo lineal multivariante tampoco logra sacarle ventaja a las exógenas (lo retomamos en el punto 8). La señal predecible de `wind_avg` a esta escala vive casi por completo en su propia estructura autorregresiva y estacional; sumar atributos meteorológicos no la incrementa, y en un modelo de alta capacidad como la LSTM incluso la perjudica.

Lección metodológica: más variables de entrada no implican mejor pronóstico. La `val_loss` más baja podría habernos engañado si nos quedáramos ahí; es **la evaluación honesta sobre el test** (la misma metodología de windowing que venimos aplicando) la que revela que las exógenas no ayudan. Por eso, para la comparación final del punto 8 conservamos la **LSTM univariante** como representante de las redes neuronales.

1.9 Comparación de modelos y elección

Reunimos los cinco modelos bajo la **misma metodología de evaluación** (walk-forward por horizonte sobre el 20 % final, métricas en kt) para una comparación justa:

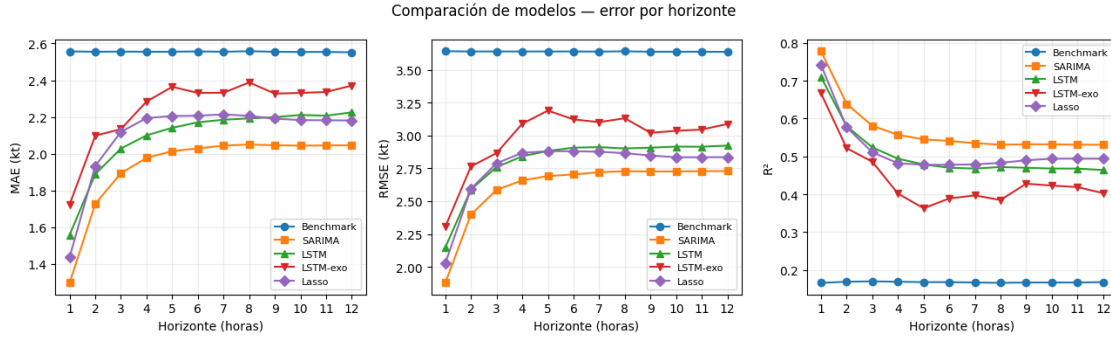
- **Benchmark:** persistencia estacional (punto 3),
- **SARIMA**(2,0,1)(1,0,1,24) (punto 4),
- **LSTM** simple univariante (punto 6),
- **LSTM-exo** con cinco variables exógenas (punto 6),
- **LassoCV** del primer entregable.

1.9.1 Re-evaluación del LassoCV bajo el mismo esquema

En el punto 5 dejamos pendiente comparar el LassoCV del entregable 1: allí se evaluó con validación cruzada sobre todo el dataset y sus cifras no eran comparables con el walk-forward de esta entrega. Acá saldamos esa deuda. Re-entrenamos el mismo pipeline (`StandardScaler` → `LassoCV` con `TimeSeriesSplit` para elegir α) sobre el 80 % inicial y lo evaluamos sobre el 20 % final, horizonte por horizonte.

Tres aclaraciones:

- **Es uno de los dos modelos con variables exógenas** (junto con la LSTM-exo). Usa 24 lags de `wind_avg` más las mismas cinco exógenas (`wind_max`, `temperature`, `rh`, `mslp`, `wind_direction`) y la hora codificada de forma cíclica, todas conocidas en el origen del pronóstico. El benchmark, SARIMA y la LSTM simple son univariantes (solo `wind_avg`). La comparación responde entonces una pregunta concreta: **¿la información exógena justifica su costo frente a los modelos univariantes?**
- **Direct multi-step.** Igual que la LSTM, ajustamos un modelo por horizonte (`objetivo = wind_avg.shift(-h)`). Para no filtrar el test en el entrenamiento dejamos un `gap` de 12 h entre train y test.
- **Conteos de orígenes ligeramente distintos.** Como ya señalamos para la LSTM, el ventaneo hace que cada modelo se evalúe sobre una cantidad de orígenes algo diferente: benchmark y SARIMA sobre 1218, las dos LSTM (univariante y exógena) sobre 1170 y el Lasso entre 1217 y 1228 según el horizonte. Los sub-períodos se solapan casi por completo, así que los promedios siguen siendo comparables, pero lo dejamos asentado por honestidad metodológica.



1.9.2 Interpretación y elección del modelo

El gráfico y la tabla resumen (medias sobre los 12 horizontes) ordenan a los cinco modelos. Lecturas principales:

- **Los cuatro modelos superan al benchmark en todos los horizontes.** La persistencia estacional (RMSE 3.64 kt, R^2 0.17, plana) queda muy por debajo: las mejoras de RMSE promedio van del 18 % (LSTM-exo) al 28 % (SARIMA).
- **SARIMA es el mejor en todo el rango.** Tiene el menor RMSE y el mayor R^2 en cada horizonte (RMSE promedio 2.61 kt, R^2 0.57). Su ventaja es máxima en el corto plazo (en $h = 1$ alcanza R^2 0.78 contra 0.74 del Lasso, 0.71 de la LSTM y 0.67 de la LSTM-exo) y se mantiene arriba hasta $h = 12$.
- **Los dos modelos con exógenas no se despegan, y la LSTM-exo es la peor de las cuatro.** El Lasso (RMSE 2.76 kt) y la LSTM univariante (2.80) quedan prácticamente empatados y por debajo de SARIMA; la **LSTM-exo es la última** (2.98 kt, R^2 0.44). Es el hallazgo central, ahora por partida doble: **ni el modelo lineal ni la red neuronal logran convertir las cinco exógenas (wind_max, temperatura, humedad, presión, dirección) en mejor pronóstico.** En la red, además, son contraproducentes: bajan la `val_loss` pero suben el error de test (sobreajuste). La estructura autorregresiva y estacional de `wind_avg` ya contiene casi toda la señal predecible a esta resolución.
- **A horizontes largos los modelos convergen** (~2.7–2.9 kt de RMSE en $h = 12$): cuando el corto plazo deja de informar, todos se apoyan en el ciclo diario y las diferencias se achican.

¿Qué modelo elegiríamos para una fase operativa? **SARIMA.** Las razones:

1. **Mejor desempeño en todos los horizontes**, con la mayor ventaja justo donde más se usa un pronóstico de viento: las próximas horas.
2. **Robustez operativa.** Es univariante: solo necesita el histórico de `wind_avg`. No depende de cinco sensores exógenos que pueden fallar o llegar tarde y, de hecho, los dos modelos que sí los usan (Lasso y LSTM-exo) no logran sacarles ventaja; en la red incluso degradan el resultado. Esa dependencia extra no se justifica.
3. **Parsimonia e interpretabilidad.** Pocos parámetros, ajuste rápido (re-fiteable de forma periódica en operación) e intervalos de predicción nativos. Con la salvedad del punto 4: los residuos no son gaussianos, así que esos intervalos subestiman los eventos de viento extremo.
4. **Las redes neuronales, las más complejas** (entrenamiento, sensibilidad a la semilla, menor interpretabilidad), no compensan: la univariante empatara al Lasso y queda por debajo de

SARIMA, y agregarle exógenas la empeora.

En síntesis, para esta serie y este horizonte el modelo estocástico ofrece la mejor relación desempeño/simplicidad/robustez.